A PROJECT REPORT

on

KELVIN: AN AI-ASSISTED DOMAIN-SPECIFIC LANGUAGE COMPILER FOR IoT AUTOMATION

Submitted for

BCSE307P – Compiler Design Lab

Assessment 9 – Project Review II

Submitted by

Name	Register Number
R. Sanjay	24BCI0181
Settipalli Tejovardhan	24BCI0196
Ashutosh	23BCI0117

Under the guidance of

Prof. _____

VELLORE INSTITUTE OF TECHNOLOGY

Vellore – 632014, Tamil Nadu, India

October 2026

ABSTRACT

Most home automation is written as small trigger–action rules, for example “if it is hot and the door is open, switch on the fan”. The people who want these rules are usually not programmers, and the AI chat tools they might turn to can return rules that look right but are not. In this project we built Kelvin, a compiler for a small rule language for IoT devices, with an AI assistant placed in front of it. A user types or speaks a rule in English, and a free language model (a local Qwen3 model through Ollama, or Groq or Gemini) turns it into our language. Nothing is accepted until our own compiler has checked it. The compiler is written by hand in C++ and covers all seven phases taught in the course, from a DFA-based lexer and an LL(1) parser to a CFG-based optimizer and code generation for C, Arduino, Python and JavaScript. It also warns about rules that can never run or that fight over the same device. On our 13 test programs it compiles each one in under 0.6 ms, removes up to 52% of the intermediate code, and passes all 145 automated tests.

Keywords: compiler design, domain-specific language, IoT automation, LL(1) parsing, rule conflict detection, dataflow analysis, code generation, large language models.

TABLE OF CONTENTS

ABSTRACT i
1. INTRODUCTION 1
1.1 Motivation 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Language and Compiler 2
1.5 Organisation of the Report 2
2. RELATED WORK 3
2.1 Parsing and Grammar Analysis 3
2.2 Control Flow, Dataflow and Intermediate Representations 3
2.3 Domain-Specific Languages 3
2.4 Trigger–Action Programming and IoT Safety 3
2.5 Natural Language to Code 4
2.6 Research Gap 5
3. PROPOSED METHODOLOGY 6
3.1 Architecture 6
3.2 Language Design 6
3.3 Compiler Phases 7
3.4 AI Assistant and Verification Gate 8
3.5 Web Studio and Digital Twin 9
3.6 Tools and Technologies 10
4. RESULTS AND DISCUSSION 11
4.1 Test Cases 11
4.2 Sample Outputs 12
4.3 Error Handling 15
4.4 Performance 16
4.5 Novelty 18
4.6 Limitations 18
5. CONCLUSION AND FUTURE WORK 20
REFERENCES 21

1. INTRODUCTION

1.1 Motivation

Almost every smart device we use at home is driven by short rules: when the temperature goes above a limit, start the fan; when motion is seen at night, switch on the porch light. Platforms such as IFTTT have shown that ordinary users are happy to write these rules themselves [1], [2]. The same studies also show that users often get the behaviour of their rules wrong, especially when two rules touch the same device [3], [4].

Today the obvious way for a non-programmer to write such a rule is to ask an AI chatbot. We tried this while planning the project and found the answers convincing but not always correct. Language models trained on code are known to produce output that looks fine and still fails [5]. A wrong rule in a real house is not just a bug: it can switch off a heater in winter or leave a door unlocked [6], [7]. This is the gap our project targets. We wanted the convenience of typing a rule in English, with a real compiler, rather than the AI, making the final decision.

1.2 Problem Statement

The task given to us was to build an AI-assisted compiler for a domain-specific language (DSL) for IoT automation. A sentence such as *"If the temperature is above 30 and the door is open, turn on the fan"* should become the DSL rule `IF temperature > 30 AND door == OPEN THEN turn_on(fan)`, and that rule should then pass through lexical analysis, parsing, semantic analysis, intermediate representation, optimization and code generation.

1.3 Objectives

1. Design a small rule language for sensors, devices and actions, with a grammar that is LL(1).
2. Write all seven compiler phases ourselves in C++17, without tools such as Lex, Yacc or ANTLR.
3. Go past type checking and detect rules that can never fire, rules that always fire, and rules that conflict with each other.
4. Optimize the intermediate code using basic blocks, a control-flow graph and dataflow analysis.
5. Generate C, Arduino, Python and JavaScript from the same intermediate code.

6. Use free AI models to convert English to the DSL, and accept their output only after our compiler has verified it.
7. Build a web interface that shows every phase and simulates the compiled rules in a virtual smart home.

1.4 Scope of the Language and Compiler

The language has numeric sensors (`NUMBER`) and on/off style sensors (`STATE`, with the values ON, OFF, OPEN, CLOSED, TRUE and FALSE), device declarations, and rules of the form `IF … THEN …` with any number of `ELSE IF` branches and an optional `ELSE`. Conditions can use comparisons, `AND`, `OR`, `NOT`, brackets and arithmetic. There are four built-in actions: `turn_on`, `turn_off`, `toggle` and `set_level`. All rules read the same snapshot of sensor values, similar to one scan cycle of a PLC.

We left out timers, schedules, variables that keep their value between runs, and user-defined functions. The Arduino output is complete as a sketch, but the functions that read real sensors are left as stubs to be wired to actual pins.

1.5 Organisation of the Report

Chapter 2 discusses earlier work that shaped our design. Chapter 3 explains the architecture, the language and each compiler phase, along with the tools we used. Chapter 4 presents our test cases, sample outputs, error handling, timing measurements and the limitations we know about. Chapter 5 concludes the report and lists what we would like to add next.

2. RELATED WORK

We read twenty papers while designing the project. We have grouped them below by the part of our system they influenced, and Table 1 at the end of the chapter lists all of them.

2.1 Parsing and Grammar Analysis

Knuth's paper on LR parsing [8] introduced the idea of deciding a grammar's parsing strategy from its structure, and it is still the basis of how parsing is taught. Parser generators such as ANTLR [9] and its later LL(*) algorithm [10] take a grammar and produce a parser automatically. We chose not to use a generator, because writing the recursive-descent functions ourselves made each step of parsing visible. To keep the benefit of formal checking, our compiler stores its own grammar as data, computes FIRST and FOLLOW sets and the LL(1) table, and confirms that no table cell has two entries.

2.2 Control Flow, Dataflow and Intermediate Representations

Allen [11] described how to split a program into basic blocks and analyse the flow between them, and Kildall [12] gave the iterative framework for dataflow problems. Our optimizer follows both directly: it builds a control-flow graph and computes liveness and available expressions by iterating to a fixed point. Cytron *et al.* [13] introduced SSA form, which we did not need for programs this small but plan to try later. LLVM [14] showed how one intermediate representation can serve many targets; our three-address code plays the same role for four back ends.

2.3 Domain-Specific Languages

Mernik, Heering and Sloane [15] discuss when a separate language is worth building and how to design one, and van Deursen, Klint and Visser [16] survey a large number of DSLs. Two points from these papers guided us: keep the notation close to how people already describe the problem, and give the language its own parser so that errors can be reported in the user's terms. Our rules therefore read almost like the English they come from.

2.4 Trigger–Action Programming and IoT Safety

Ur *et al.* showed that non-programmers can write useful smart-home rules [1] and studied 200,000 real IFTTT recipes [2]. Huang and Cakmak [3] found that users' understanding of when a rule fires is often inaccurate, and Brackenbury *et al.* [4] catalogued the kinds of bugs that appear in such rules, including conflicting actions on the same device. Soteria [6]

checks IoT apps for unsafe states using model checking, and Ding and Hu [7] showed that devices can affect each other through the physical environment. These works convinced us to add rule analysis to the semantic phase, so that conflicts and unreachable branches are reported at compile time, together with a sample input that shows the problem.

2.5 Natural Language to Code

Desai *et al.* [17] generated DSL programs from natural language using a grammar and a ranking model, NL2Bash [18] translated English into shell commands, and Yin and Neubig [19] generated code by predicting syntax-tree actions. Chen *et al.* [5] evaluated large language models on programming tasks and showed that a large share of generated programs fail their tests. Synchromesh [20] improves reliability by restricting generation to valid syntax. Our approach is simpler but strict: whatever the model returns is compiled, the compiler's error messages are sent back for one repair attempt, and a meaning check rejects answers that compile but leave out part of the user's sentence.

Table 1: Summary of the related work

No.	Work (year)	Main idea	How it relates to our project
[1]	Ur et al. (2014)	End users writing smart-home rules	Shape of our rules and target users
[2]	Ur et al. (2016)	Study of 200,000 IFTTT recipes	Realistic rule patterns for examples
[3]	Huang & Cakmak (2015)	Users misjudge when rules fire	Need for clear diagnostics
[4]	Brackenbury et al. (2019)	Bugs in trigger-action rules	Conflict and reachability warnings
[5]	Chen et al. (2021)	Evaluating code-generating LLMs	Why AI output must be verified
[6]	Celik et al. (2018)	Model checking IoT apps (Soteria)	Static safety checks before deployment
[7]	Ding & Hu (2018)	Physical interaction between devices	Detecting devices driven both ways
[8]	Knuth (1965)	LR parsing theory	Background for grammar analysis
[9]	Parr & Quong (1995)	ANTLR LL(k) parser generator	We hand-write and verify LL(1) instead
[10]	Parr & Fisher (2011)	LL(*) parsing	Comparison for our LL(1) grammar
[11]	Allen (1970)	Control-flow analysis	Basic blocks and CFG
[12]	Kildall (1973)	Iterative dataflow analysis	Liveness and available expressions
[13]	Cytron et al. (1991)	Static single assignment form	Planned future improvement
[14]	Lattner & Adve (2004)	One IR for many targets (LLVM)	One TAC for four back ends
[15]	Mernik et al. (2005)	When and how to build a DSL	Language design decisions

No.	Work (year)	Main idea	How it relates to our project
[16]	van Deursen et al. (2000)	Survey of DSLs	Notation close to the domain
[17]	Desai et al. (2016)	Natural language to DSL	Our English-to-DSL step
[18]	Lin et al. (2018)	English to shell commands	Natural-language interface design
[19]	Yin & Neubig (2017)	Syntax-driven code generation	Grammar-aware generation
[20]	Poesia et al. (2022)	Grammar-constrained LLM output	Future work; we verify after generation

2.6 Research Gap

Existing rule platforms accept whatever the user writes, safety checkers analyse finished apps rather than a language with a grammar, and natural-language code tools mostly trust the model. We did not find a system that combines a small rule language with a verified LL(1) grammar, compile-time conflict detection with example inputs, an optimizing back end for both microcontrollers and scripting languages, and an AI front end whose every answer is checked by the compiler. Kelvin is our attempt to bring these pieces together.

3. PROPOSED METHODOLOGY

3.1 Architecture

Figure 1 shows the overall design. On the left, a sentence typed or spoken by the user goes to the AI assistant, and its answer has to pass a verification gate before it is used. The DSL program then moves through the front end (lexer, parser, syntax tree, semantic and rule analysis), the middle end (three-address code, control-flow graph and optimizer) and the back end (code generators and their runtime libraries). The bottom row shows the web layer: a small Python server makes the compiler available to the browser, where the studio displays every phase and a digital twin runs the optimized code. Figure 2 summarises what each phase produces.

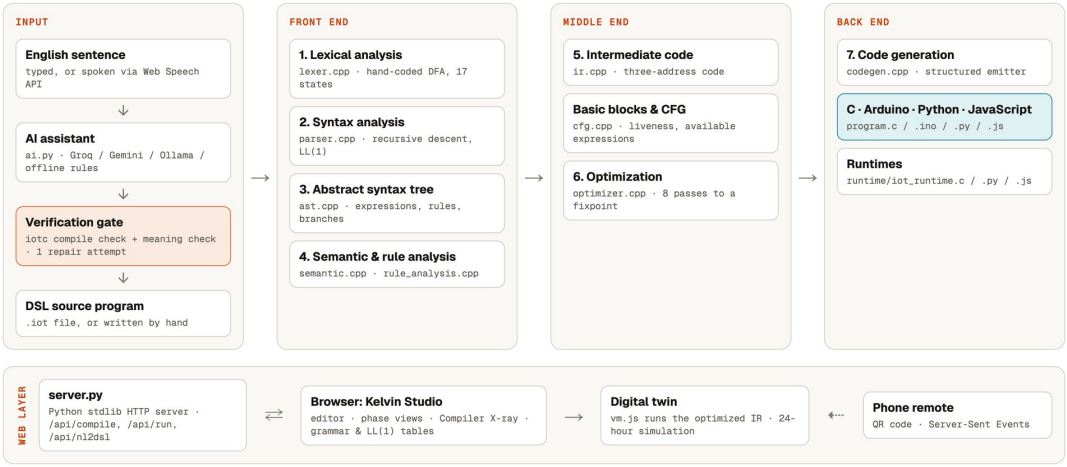


Figure 1: System architecture of Kelvin

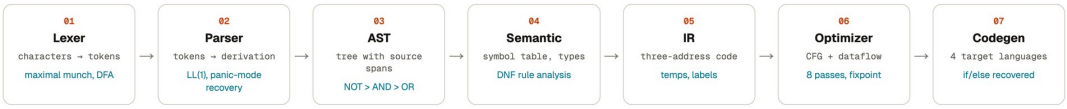


Figure 2: The seven compiler phases and their outputs

3.2 Language Design

The grammar is shown below in EBNF. Inside the compiler the same grammar is stored in BNF form, with 61 productions, 29 nonterminals and 33 terminals. The compiler computes the FIRST and FOLLOW sets and fills the LL(1) parse table, and it reports 0 conflicts. Operator precedence runs from unary minus (highest), through `*` `/`, `+` `-` and comparisons, to `NOT`, `AND` and finally `OR` (lowest). Conditions and arithmetic share one expression

grammar, the way C does, which also avoided an LL(1) conflict between a bracketed condition and a bracketed calculation.

```
program     → { declaration | rule } EOF
declaration → SENSOR IDENT ':' ( NUMBER | STATE )
            | DEVICE IDENT { ',' IDENT }
rule        → IF expr THEN actions { ELSE IF expr THEN actions } [ ELSE actions ]
actions     → action { action }
action      → IDENT '(' [ expr { ',' expr } ] ')'
expr        → and_expr { OR and_expr }
and_expr    → not_expr { AND not_expr }
not_expr    → NOT not_expr | rel_expr
rel_expr    → arith [ relop arith ]
arith       → term { ( '+' | '-' ) term }
term        → unary { ( '*' | '/' ) unary }
unary       → '-' unary | primary
primary     → NUMBER_LITERAL | IDENT | ON | OFF | OPEN | CLOSED | TRUE | FALSE
            | '(' expr ')'
relop       → '>' | '<' | '>=' | '<=' | '==' | '!='
```

3.3 Compiler Phases

Phase 1: Lexical analysis

The scanner is a hand-coded finite automaton with 17 states. It reads one character at a time and always takes the longest token it can (maximal munch), so >= is read as one operator and not as > followed by =. Keywords are recognised after a whole word has been read. Each token keeps its line and column. When the scanner meets a character it does not know, or a single = where == was meant, it reports the problem with a suggested correction and carries on.

Phases 2 and 3: Syntax analysis and the syntax tree

The parser is recursive descent, with one C++ function per grammar rule, for example `parseRule`, `parseExpr`, `parseAnd` and `parseRelational`. Because the grammar is LL(1), the next token is always enough to choose what to do. When a syntax error is found, the parser skips ahead to the next `IF`, `SENSOR` or `DEVICE` (panic-mode recovery), which lets it report several errors in a single run instead of stopping at the first. The syntax tree uses one family of expression nodes (binary, unary and operand) for both conditions and arithmetic, and every node remembers where it came from in the source.

Phase 4: Semantic analysis and rule analysis

The semantic phase builds the symbol table, checks that each name is declared exactly once, and works out a type for every expression: NUMBER, STATE or BOOL. It rejects comparisons between incompatible types and wrong arguments to actions. A misspelt name

gets a *did you mean* suggestion based on Levenshtein distance, and many errors come with a fix that can be applied with one click in the studio.

We then added rule analysis, which is the part we are most proud of. Each condition is rewritten into disjunctive normal form, and each AND-group is checked using number intervals for numeric sensors and sets of possible values for state sensors. This is enough to report conditions that can never be true, conditions that are always true, `ELSE IF` branches that can never be reached, and two rules that set the same device to different states for the same input. For a conflict, the warning includes an example, such as *temperature = 31 and door = OPEN*.

Phase 5: Intermediate code

The intermediate form is three-address code with temporaries (`t1`, `t2`, …), labels and the instructions `IF_FALSE … GOTO`, `GOTO` and `CALL`. An `ELSE IF` chain becomes a sequence of tests and labels. Every instruction keeps the source position it was created from, which the studio uses to highlight matching lines across all views.

Phase 6: Optimization

The optimizer first splits the code into basic blocks and builds a control-flow graph. It then computes two dataflow analyses: liveness, which works backwards, and available expressions, which works forwards [12]. Eight passes are run again and again until the code stops changing: constant folding, algebraic simplification, common sub-expression elimination, copy propagation, removal of constant branches, removal of unreachable code, jump optimization and dead-code elimination. The optimizer also records a snapshot after each pass, so the studio can replay its work step by step.

Phase 7: Code generation

A single structured emitter turns the optimized code back into `if / else` blocks and passes them to one of four back ends. The C back end links with a small runtime library; the Arduino back end produces a sketch with `setup()`, `loop()` and a pin table; the Python and JavaScript back ends come with their own runtimes. All four print their results in the same format, which made it easy to test them against each other. The compiler can also translate a program back into plain English, which we use for the *Explain* feature.

3.4 AI Assistant and Verification Gate

The AI module supports three free services: Groq, Google Gemini and a local Ollama server, which in our setup runs the Qwen3 8B model. If none is available, a rule-based

converter we wrote handles the common sentence patterns. The prompt contains a short language reference and the names already declared in the editor. When an answer comes back, it is compiled. If the compiler reports errors, they are sent back to the model once with a request to correct the answer.

During testing we noticed that the local model sometimes dropped part of a sentence. For example, *"above 30 and the door is open"* came back as `> 3`. The result still compiled, so the first version of our gate accepted it. We fixed this by adding a meaning check: every number in the sentence must appear in the program, and every declared sensor must be used by a rule. If the answer still fails after the repair attempt, the rule-based converter is used instead, and the interface says so.

3.5 Web Studio and Digital Twin

The studio is the interface we use for demonstrations (Figure 3). It has an editor that compiles while you type, a strip showing the status of each phase, and tabs for tokens, the syntax tree, the grammar tables, symbols, intermediate code, generated code and problems. The *Compiler X-ray* replays a compilation using traces recorded by the C++ compiler, so the automaton, the parser's call stack and each optimizer pass can be watched in order. The digital twin runs the optimized code in a small virtual machine and animates devices on a house plan. A 24-hour simulation moves the sensors along daily curves, and a phone can act as a sensor panel by scanning a QR code.

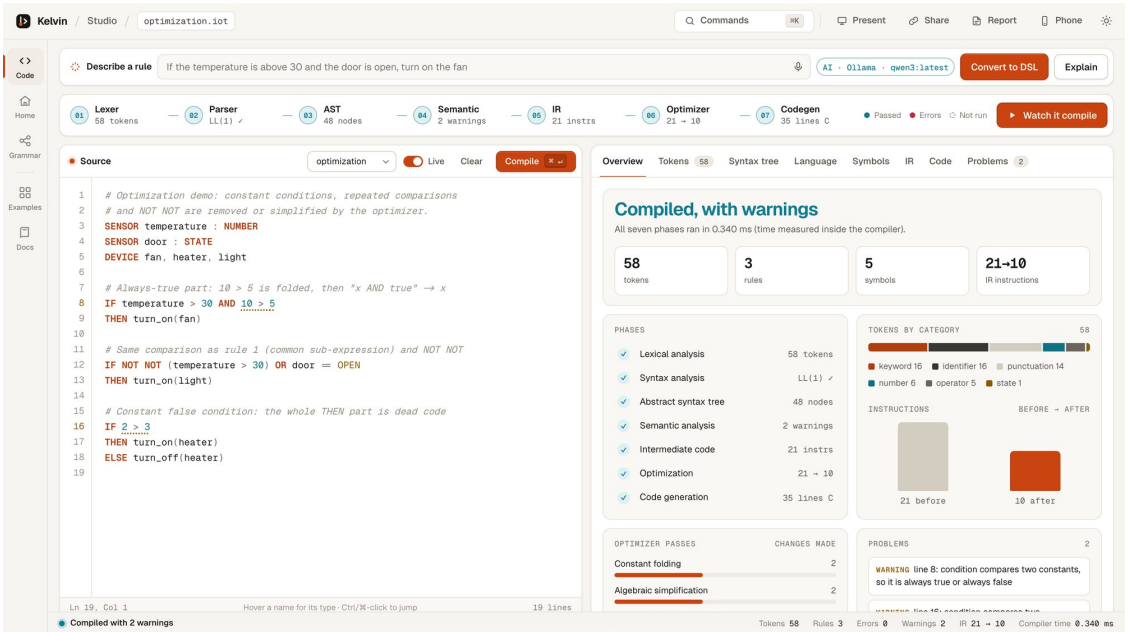


Figure 3: The studio showing the overview of a compilation (example 07)

3.6 Tools and Technologies

Table 2: Tools and technologies used

Component	Technology	Remarks
Compiler (iotc)	C++17 (Apple clang 21 / g++)	4,865 lines in 28 files; no external libraries
Build	GNU Make	make, make run, make remote, make test
Runtimes	C99, Python 3, Node.js	Same output format for every target
Web server	Python 3.14 standard library	http.server, JSON API, Server-Sent Events
Web interface	HTML, CSS, JavaScript	No frameworks; fonts stored locally; works offline
AI models	Ollama (Qwen3 8B), Groq, Gemini	Free services; rule-based fallback
Testing	Shell script, Python unittest	126 compiler checks and 19 unit tests
Version control	Git and GitHub	Private repository
Test machine	Apple M3, 16 GB RAM, macOS 27	Used for all measurements

4. RESULTS AND DISCUSSION

4.1 Test Cases

We tested the project at two levels. A shell script, `tests/run_tests.sh`, runs 126 checks against the compiler: it compiles programs, compares messages, builds the generated C with warnings treated as errors, and runs the result. Python `unittest` adds 19 tests for the web server, the phone remote's access rules and the AI checks. All 145 tests pass. Table 3 groups the compiler checks by area, and Table 4 lists the example programs we use as end-to-end test cases.

Table 3: Compiler checks grouped by area (all passing)

Area	Checks	What is checked
Example programs	7	Each example compiles, or fails with the expected message
Lexer and parser	9	Tokens, positions, precedence and error recovery
Semantic analysis	12	Undeclared names, type errors, argument counts, suggestions
IR and optimization	5	Three-address code and instruction counts
Code generation	6	Generated C builds with -Werror and sets devices correctly
Arithmetic and ELSE IF	11	Arithmetic in conditions and actions; ELSE IF chains
Rule analysis	10	Impossible, always-true, unreachable and conflicting rules
CFG and dataflow	4	Basic blocks, edges and liveness
Generated C for ELSE IF	7	Run-time behaviour of the generated C
New example programs	16	Security, air quality, kitchen and office examples
Grammar, English, fixes, traces	18	LL(1) table, English output, quick fixes, traces
Cross-target equivalence	20	C, Python and JavaScript agree; Arduino sketches compile
JSON output	1	Machine-readable output is valid
Python unit tests	19	Remote access rules, run endpoint, AI checks

Table 4: Example programs used as test cases

Program	What it tests	Expected result	Status
01 basic	Problem-statement example	Fan on when hot and door open	Pass
02 smart home	Precedence, NOT, set_level	Compiles with 3 rules	Pass
03 greenhouse	NOT with states, levels	Compiles with 2 rules	Pass
04 lexical error	Bad characters, '=' for '=='	4 lexical errors with fixes	Pass
05 syntax error	Lowercase keyword, missing THEN and ')'	3 syntax errors reported	Pass
06 semantic error	Undeclared names, type and arity errors	6 semantic errors	Pass

Program	What it tests	Expected result	Status
07 optimization	Constant conditions, repeated tests	21 → 10 instructions	Pass
08 rule analysis	Impossible, unreachable, conflicting rules	4 warnings with examples	Pass
09 thermostat	ELSE IF chain, arithmetic	Heater level (22 − t) × 10	Pass
10 home security	Brackets, ELSE IF, several actions	Alarm on intrusion	Pass
11 air quality	Arithmetic inside an action	Vent level (co2 − 800) / 4	Pass
12 kitchen safety	OR conditions, safety shut-off	Gas valve closed on a leak	Pass
13 office energy	NOT, long AND chains	Lights and AC follow people	Pass

4.2 Sample Outputs

The outputs below were produced by our compiler for the example from the problem statement (`examples/01_basic.iot`).

```
SENSOR temperature : NUMBER
SENSOR door : STATE
DEVICE fan
IF temperature > 30 AND door == OPEN
THEN turn_on(fan)
```

Phase 1 output (the first eight of 23 tokens):

```
#    LINE:COL  TOKEN         LEXEME
1    2:1       SENSOR        SENSOR
2    2:8       IDENTIFIER    temperature
3    2:20      COLON         :
4    2:22      TYPE_NUMBER   NUMBER
5    3:1       SENSOR        SENSOR
6    3:8       IDENTIFIER    door
7    3:13      COLON         :
8    3:15      TYPE_STATE    STATE
```

Phases 2 and 3 output (the rule part of the syntax tree), also drawn by the studio in Figure 4:

```
Rule
├── Condition
│   └── AND
│       ├── Comparison: >
│       │   ├── Identifier: temperature
│       │   └── Number: 30
│       └── Comparison: ==
│           ├── Identifier: door
│           └── State: OPEN
└── Then
    └── Action: turn_on
        └── Identifier: fan
```

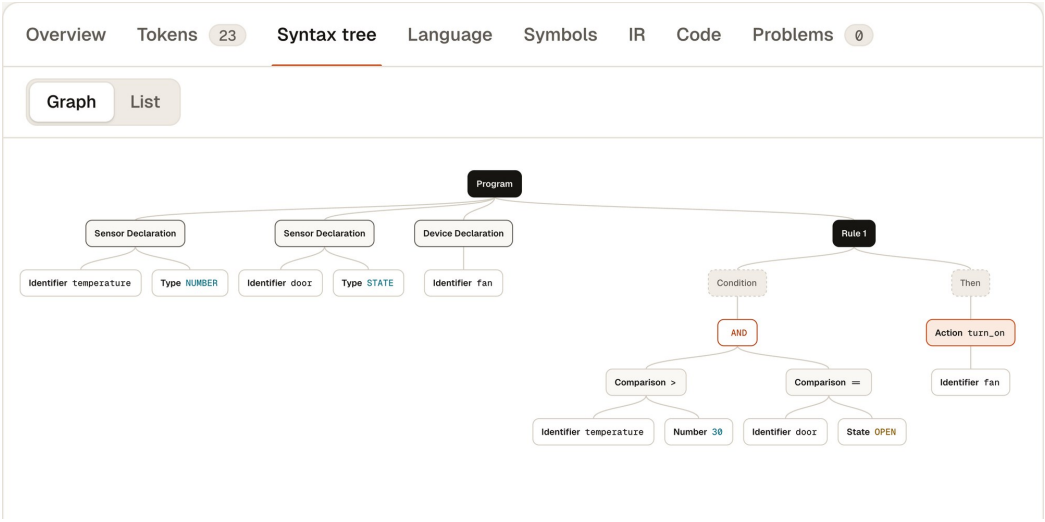


Figure 4: Syntax tree of the basic example in the studio

Phases 5 and 6 output. For `examples/07_optimization.iot` we deliberately wrote a constant condition, a repeated comparison, a double `NOT` and a branch that can never run. The optimizer reduces 21 instructions to 10:

```
BEFORE (21 instructions)                AFTER (10 instructions)
# Rule 1                                # Rule 1
    t1 = temperature > 30                   t1 = temperature > 30
    t2 = 10 > 5                             IF_FALSE t1 GOTO L1
    t3 = t1 AND t2                          CALL turn_on(fan)
    IF_FALSE t3 GOTO L1                 L1:
    CALL turn_on(fan)                   # Rule 2
L1:                                         t7 = door == OPEN
# Rule 2                                    t8 = t1 OR t7
    t4 = temperature > 30                   IF_FALSE t8 GOTO L2
    t5 = NOT t4                             CALL turn_on(light)
    t6 = NOT t5                         L2:
    t7 = door == OPEN                   # Rule 3
    t8 = t6 OR t7                           CALL turn_off(heater)
    IF_FALSE t8 GOTO L2
    CALL turn_on(light)
L2:
# Rule 3
    t9 = 2 > 3
    IF_FALSE t9 GOTO L3
    CALL turn_on(heater)
    GOTO L4
L3:
    CALL turn_off(heater)
L4:
```

The passes that made changes were constant folding (2 times), algebraic simplification (2), common sub-expression elimination (1), copy propagation (5), constant-branch removal (1), unreachable-code removal (2), jump optimization (3) and dead-code elimination (1).

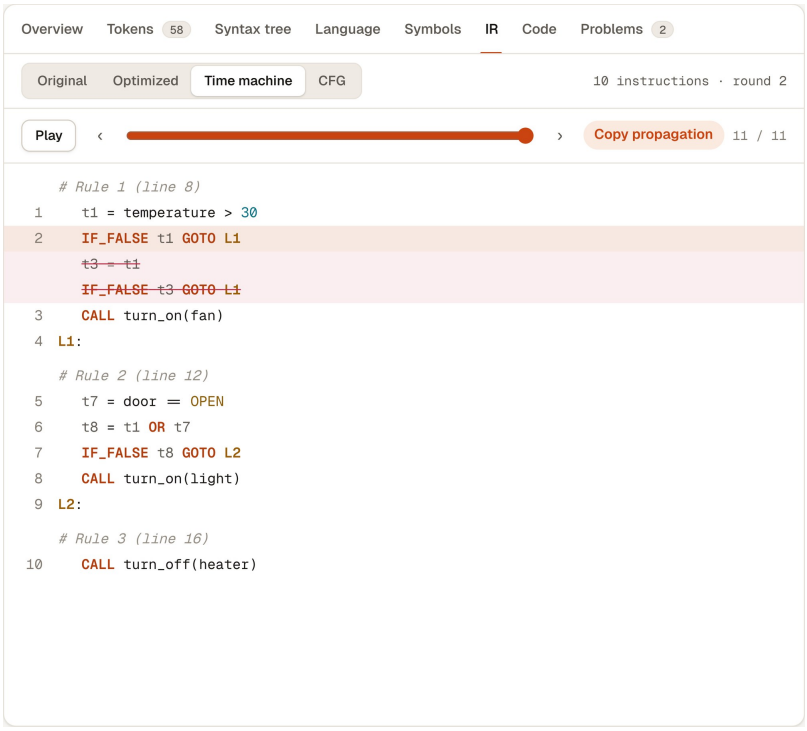


Figure 5: Optimizer replay in the studio; struck-out lines were removed

Phase 7 output (generated C for the basic example):

```
void evaluate_rules(void)
{
    /* Read every sensor once, so all rules see the same snapshot */
    double temperature = read_number("temperature");
    state_t door = read_state("door");


    /* Rule 1 (source line 6) */
    bool t1 = temperature > 30;
    bool t2 = door == STATE_OPEN;
    bool t3 = t1 && t2;
    if (t3) {
        turn_on(DEV_FAN);
    }
}
```

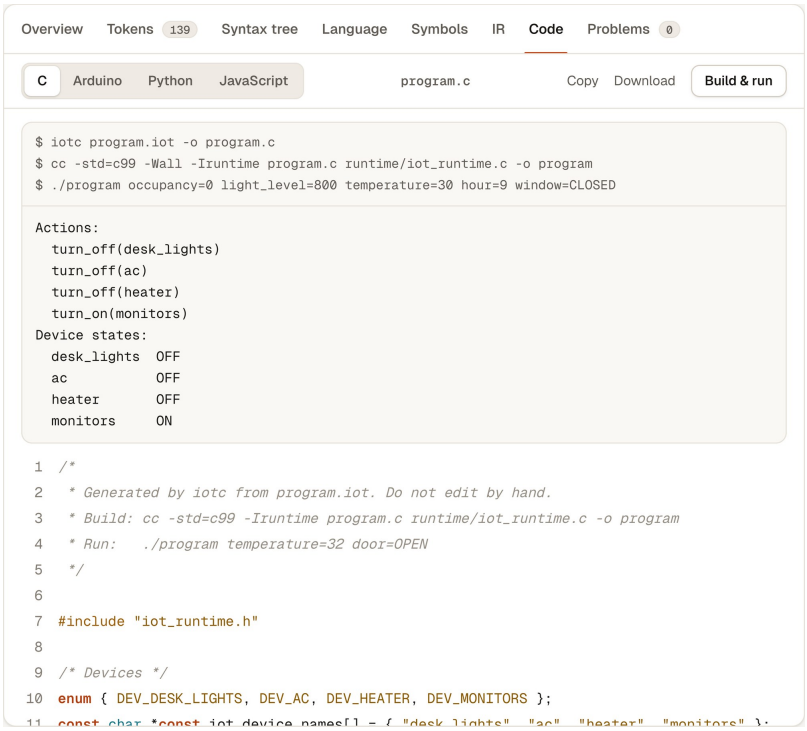


Figure 6: Generated C for example 13, compiled and run from the studio

4.3 Error Handling

We treated error messages as a feature rather than an afterthought. Each phase reports its own errors with a line and column, continues where it safely can, and attaches a fix when the correction is clear. Table 5 lists real messages from our error examples, and Figures 7 and 8 show how they appear in the studio.

Table 5: Messages produced by the compiler

Phase	Message (shortened)	Quick fix
Lexical	unexpected character '@'	Remove '@'
Lexical	unexpected '=': assignment is not supported, did you mean '=='?	Replace with '=='
Syntax	expected 'IF', 'SENSOR' or 'DEVICE' but found 'if' (keywords are uppercase)	Change to 'IF'
Syntax	expected 'THEN' after the condition but found 'turn_on'	Insert 'THEN'
Syntax	expected ',' or ')' in the argument list of 'turn_on' but found end of file	Insert ')'
Semantic	'fan' is not a registered device	Declare 'DEVICE fan'
Semantic	'temprature' is not a declared sensor (did you mean 'temperature'?)	Change the name
Semantic	type mismatch: cannot compare STATE sensor 'door' with number 5	–
Semantic	'set_level' expects 2 arguments but got 1	–
Rule analysis	this condition can never be true, so its actions never run	–

Phase	Message (shortened)	Quick fix
Rule analysis	this ELSE IF can never run: an earlier condition is already true	–
Rule analysis	conflict: rule 4 turns 'fan' off while rule 3 turns it on, e.g. when temperature = 31 and door = OPEN	–

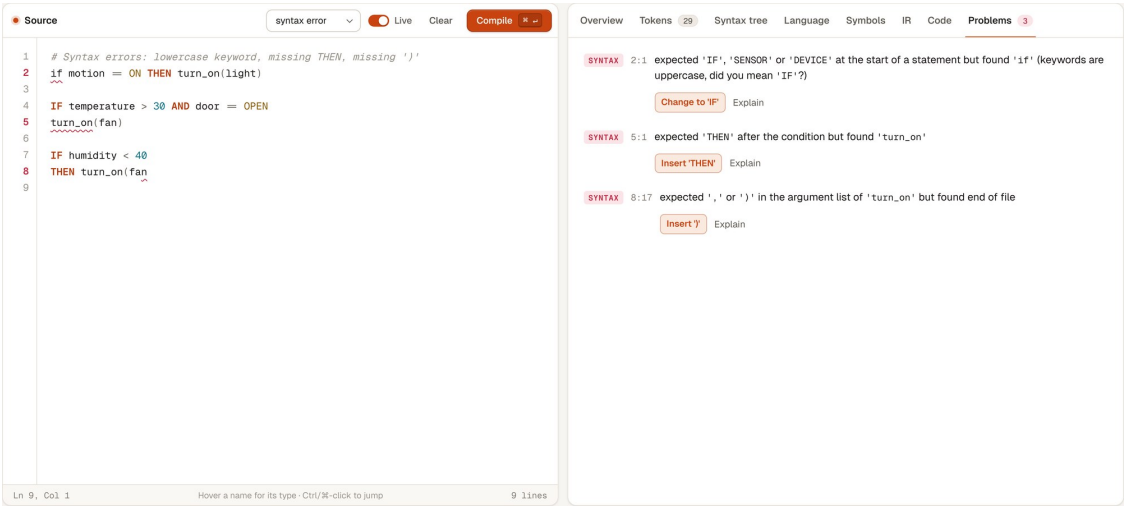


Figure 7: Three syntax errors reported in one run, each with a fix (example 05)

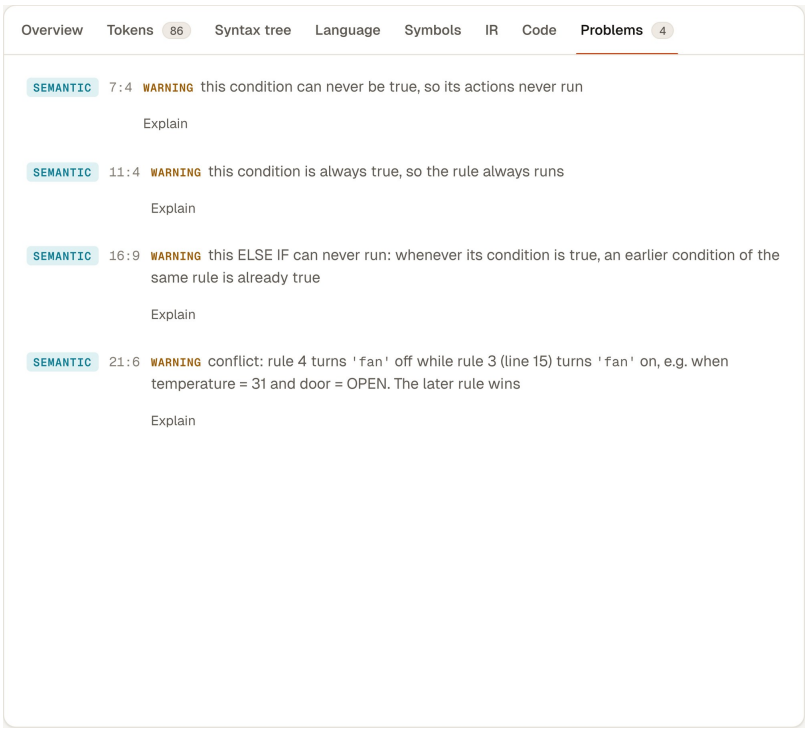


Figure 8: Rule-analysis warnings with example inputs (example 08)

4.4 Performance

All timings were measured on our test machine, an Apple M3 laptop with 16 GB of RAM, with the compiler built using `-O2`. *Compiler time* is measured inside `iotc` and covers all

seven phases. *End-to-end time* also includes starting the process and writing the JSON output. Every value is the median of 30 runs.

Table 6: Compiler timing on the example programs

Program	Lines	Tokens	Rules	IR before → after	Compiler time (ms)	End-to-end (ms)
basic	7	23	1	6 → 6	0.118	2.46
smart home	20	87	3	27 → 26	0.317	2.83
greenhouse	13	52	2	15 → 15	0.168	2.43
lexical error	4	12	0	– (errors)	0.023	2.00
syntax error	8	29	0	– (errors)	0.045	2.08
semantic error	14	45	3	– (errors)	0.032	2.11
optimization	18	58	3	21 → 10	0.225	2.62
rule analysis	26	86	6	33 → 32	0.388	2.94
thermostat	23	96	2	33 → 33	0.230	2.66
home security	25	102	3	34 → 31	0.325	2.82
air quality	28	118	3	42 → 40	0.365	2.93
kitchen safety	25	98	3	34 → 32	0.342	3.02
office energy	29	139	4	52 → 51	0.567	4.13

Every example compiles in under 0.6 ms, and the time grows roughly in line with the number of tokens. Programs with errors finish fastest because the later phases are skipped. The optimizer removes 52% of the code in the optimization example. In the more realistic programs it removes one to three instructions, mostly repeated comparisons, and leaves programs that are already minimal unchanged, which is what we expected.

We also timed the English-to-DSL step. With the local Qwen3 8B model, one sentence took between 2.5 and 2.7 seconds once the model was loaded; the very first request took 10.4 seconds because the model had to be loaded into memory. The rule-based converter took less than 1 ms. In all our timed runs the final answer compiled without errors.

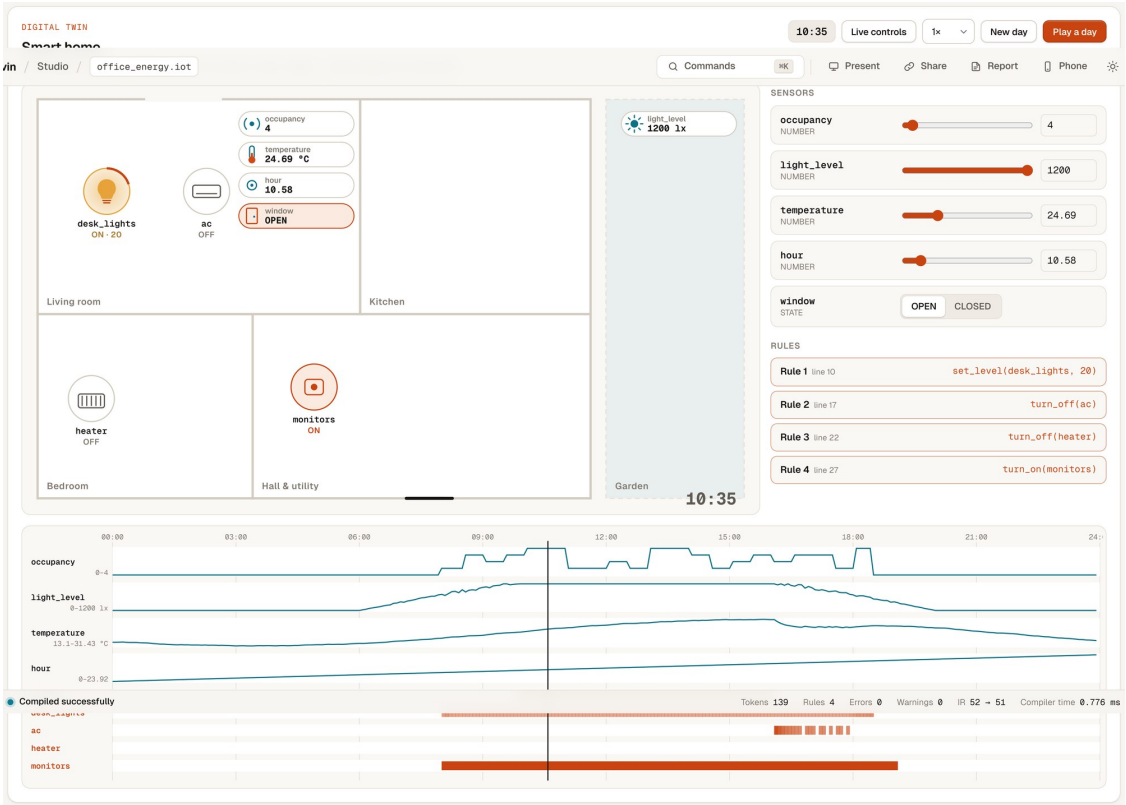


Figure 9: Digital twin running example 13 at 10:55 in a simulated day

4.5 Novelty

- The AI is never trusted on its own. Its output has to compile, pass the meaning check and, if needed, survive one repair attempt guided by the compiler's own messages.
- Conflicting and unreachable rules are found at compile time, and each warning includes a concrete input that shows the problem.
- The compiler checks its own grammar: it computes FIRST and FOLLOW sets and the LL(1) table and confirms there are no conflicts.
- One intermediate code drives four targets, and our tests confirm that the C, Python and JavaScript programs behave identically.
- The compilation can be watched: the X-ray replays the real automaton, parser calls and optimizer passes, and the digital twin runs the optimized code.

4.6 Limitations

- There are no timers or schedules, so a rule like *"turn off the light ten minutes after motion stops"* cannot be written yet.
- Rule analysis is exact when sensors are compared with constants, but it treats comparisons between two sensors, such as `temperature - outside > 5`, conservatively, so some conflicts of that kind are not reported.

- The Arduino output has been compiled against a stub header but not yet tested on a real board.
- The local model takes a few seconds per sentence on a laptop, and the meaning check catches missing numbers and unused sensors but not every possible misreading.
- The physics in the digital twin is deliberately simple and is meant for demonstration only.

5. CONCLUSION AND FUTURE WORK

We set out to build a compiler that lets people describe home automation in plain English without giving the final say to an AI model, and we have a working system that does this. All seven phases are written by hand in C++17: a finite-automaton lexer, an LL(1) recursive-descent parser with error recovery, a syntax tree with source positions, semantic analysis with rule analysis, three-address code, a CFG-based optimizer using liveness and available expressions, and code generation for four languages. The project is backed by 145 passing tests, compiles realistic programs in well under a millisecond, and can be demonstrated end to end through the studio, the X-ray and the digital twin.

Building the project taught us how closely the phases depend on each other. Keeping source positions from the lexer all the way to the generated code, for example, is what made good error messages and the linked views possible. The bug we found in the AI step also showed us that a program can be *valid* and still not *mean* what the user asked for.

Future work. We plan to:

- add timers, schedules and persistent state, together with analyses for timing conflicts;
- extend rule analysis to relations between sensors using an SMT solver, and try an SSA-based intermediate form [13];
- run the Arduino/ESP32 output on real hardware and connect it through MQTT;
- use grammar-constrained generation [20] so that the model can only produce valid DSL, and measure its accuracy on a larger set of sentences;
- carry out a small study with non-programmers, following the approach of [1] and [4].

REFERENCES

[1] B. Ur, E. McManus, M. Pak Yong Ho, and M. L. Littman, "Practical trigger-action programming in the smart home," in *Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI)*, 2014, pp. 803–812.

[2] B. Ur, M. Pak Yong Ho, S. Brawner, J. Lee, S. Mennicken, N. Picard, D. Schulze, and M. L. Littman, "Trigger-action programming in the wild: An analysis of 200,000 IFTTT recipes," in *Proc. CHI Conf. Human Factors in Computing Systems*, 2016, pp. 3227–3231.

[3] J. Huang and M. Cakmak, "Supporting mental model accuracy in trigger-action programming," in *Proc. ACM Int. Joint Conf. Pervasive and Ubiquitous Computing (UbiComp)*, 2015, pp. 215–225.

[4] W. Brackenbury, A. Deora, J. Ritchey, J. Vallee, W. He, G. Wang, M. L. Littman, and B. Ur, "How users interpret bugs in trigger-action programming," in *Proc. CHI Conf. Human Factors in Computing Systems*, 2019, pp. 1–12.

[5] M. Chen *et al.*, "Evaluating large language models trained on code," arXiv preprint arXiv:2107.03374, 2021.

[6] Z. B. Celik, P. McDaniel, and G. Tan, "Soteria: Automated IoT safety and security analysis," in *Proc. USENIX Annual Technical Conf. (ATC)*, 2018, pp. 147–158.

[7] W. Ding and H. Hu, "On the safety of IoT device physical interaction control," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2018, pp. 832–846.

[8] D. E. Knuth, "On the translation of languages from left to right," *Information and Control*, vol. 8, no. 6, pp. 607–639, 1965.

[9] T. J. Parr and R. W. Quong, "ANTLR: A predicated-LL(k) parser generator," *Software: Practice and Experience*, vol. 25, no. 7, pp. 789–810, 1995.

[10] T. Parr and K. Fisher, "LL(*): *The foundation of the ANTLR parser generator," in* Proc. 32nd ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2011, pp. 425–436.

[11] F. E. Allen, "Control flow analysis," *ACM SIGPLAN Notices*, vol. 5, no. 7, pp. 1–19, 1970.

[12] G. A. Kildall, "A unified approach to global program optimization," in *Proc. 1st ACM Symp. Principles of Programming Languages (POPL)*, 1973, pp. 194–206.

[13] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, "Efficiently computing static single assignment form and the control dependence graph," *ACM Trans. Programming Languages and Systems*, vol. 13, no. 4, pp. 451–490, 1991.

[14] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *Proc. Int. Symp. Code Generation and Optimization (CGO)*, 2004, pp. 75–86.

[15] M. Mernik, J. Heering, and A. M. Sloane, "When and how to develop domain-specific languages," *ACM Computing Surveys*, vol. 37, no. 4, pp. 316–344, 2005.

[16] A. van Deursen, P. Klint, and J. Visser, "Domain-specific languages: An annotated bibliography," *ACM SIGPLAN Notices*, vol. 35, no. 6, pp. 26–36, 2000.

[17] A. Desai, S. Gulwani, V. Hingorani, N. Jain, A. Karkare, M. Marron, S. R, and S. Roy, "Program synthesis using natural language," in *Proc. 38th Int. Conf. Software Engineering (ICSE)*, 2016, pp. 345–356.

[18] X. V. Lin, C. Wang, L. Zettlemoyer, and M. D. Ernst, "NL2Bash: A corpus and semantic parser for natural language interface to the Linux operating system," in *Proc. 11th Int. Conf. Language Resources and Evaluation (LREC)*, 2018.

[19] P. Yin and G. Neubig, "A syntactic neural model for general-purpose code generation," in *Proc. 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017, pp. 440–450.

[20] G. Poesia, O. Polozov, V. Le, A. Tiwari, G. Soares, C. Meek, and S. Gulwani, "Synchromesh: Reliable code generation from pre-trained language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.